

CLASS ACTIVITY- CASSANDRA PYTHON DRIVER

ITESO
Universidad Jesuita
de Guadalajara



Profesora: Ing. Ivanna Haro

Nombre estudiante: John Daniel Villa Velásquez

Expediente: 770304

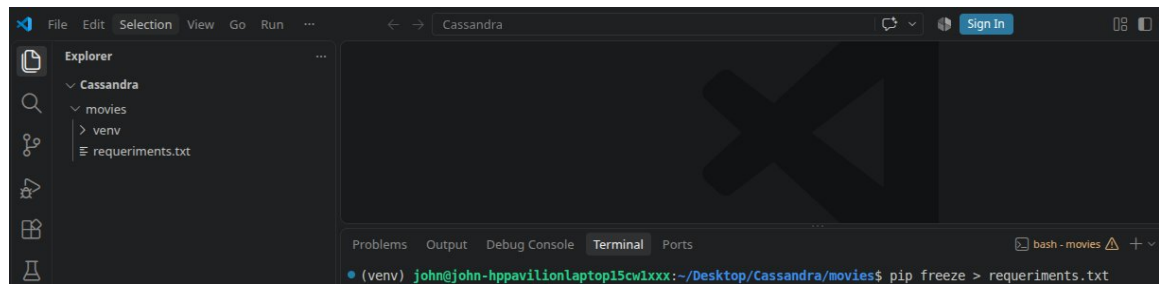
Fecha:13/09/2026

Paso 1: Crear el entorno del proyecto

Creacion del ambiente virtual de python e instalacion de la libreria cassandra driver

```
john@john-hppavilionlaptop15cw1xxx:~/Desktop/Cassandra$ mkdir movies
john@john-hppavilionlaptop15cw1xxx:~/Desktop/Cassandra$ cd movies
john@john-hppavilionlaptop15cw1xxx:~/Desktop/Cassandra/movies$ python3 -m venv venv
john@john-hppavilionlaptop15cw1xxx:~/Desktop/Cassandra/movies$ source venv/bin/activate
(venv) john@john-hppavilionlaptop15cw1xxx:~/Desktop/Cassandra/movies$ pip install cassandra-driver
Collecting cassandra-driver
  Using cached cassandra_driver-3.30.1-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (6.8 kB)
Collecting geomet>=1.1 (from cassandra-driver)
  Using cached geomet-1.1.0-py3-none-any.whl.metadata (11 kB)
Collecting Deprecated>=1.3.1 (from cassandra-driver)
  Using cached deprecated-1.3.1-py2.py3-none-any.whl.metadata (5.9 kB)
Collecting wrapt<3,>=1.10 (from Deprecated>=1.3.1->cassandra-driver)
  Downloading wrapt-2.4.1-cp314-cp314-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (7.4 kB)
Collecting click (from geomet>=1.1->cassandra-driver)
  Using cached click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Using cached cassandra_driver-3.30.1-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (3.5 MB)
Using cached deprecated-1.3.1-py2.py3-none-any.whl (11 kB)
Downloading wrapt-2.4.1-cp314-cp314-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (237 kB)
Using cached geomet-1.1.0-py3-none-any.whl (31 kB)
Using cached click-8.5.0-py3-none-any.whl (125 kB)
Installing collected packages: wrapt, click, geomet, Deprecated, cassandra-driver
Successfully installed Deprecated-1.3.1 cassandra-driver-3.30.1 click-8.5.0 geomet-1.1.0 wrapt-2.4.1
(venv) john@john-hppavilionlaptop15cw1xxx:~/Desktop/Cassandra/movies$ pip freeze
cassandra-driver==3.30.1
click==8.5.0
Deprecated==1.3.1
```

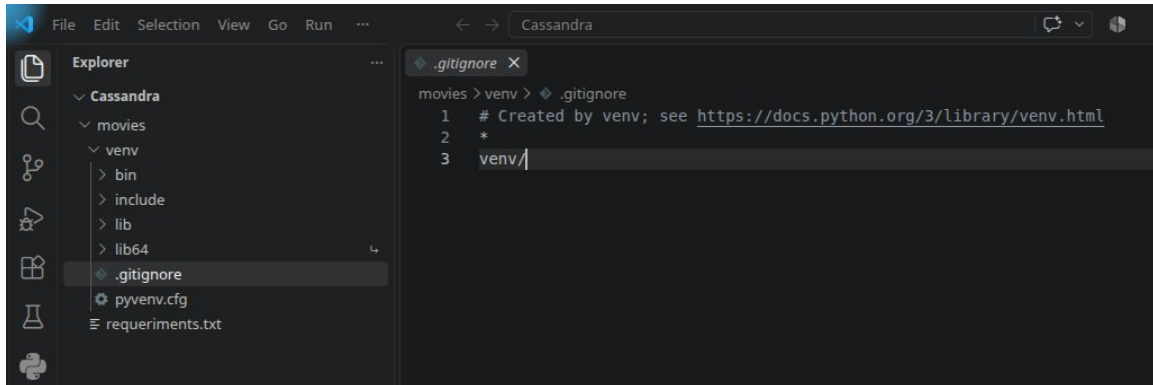
Creacion del archivo requirements



Iniciar git en el proyecto y conexion con el repositorio en git hub

```
john@john-hppavilionlaptop15cw1xxx:~/Desktop/Cassandra/movies$ git init
Iniciado repositorio Git vacío en /home/john/Desktop/Cassandra/movies/.git/
john@john-hppavilionlaptop15cw1xxx:~/Desktop/Cassandra/movies$ git branch -M main
john@john-hppavilionlaptop15cw1xxx:~/Desktop/Cassandra/movies$ git remote add origin https://github.com/JohnV-1/Class-Activity---Cassandra-Python-Driver.git
john@john-hppavilionlaptop15cw1xxx:~/Desktop/Cassandra/movies$ git add .
john@john-hppavilionlaptop15cw1xxx:~/Desktop/Cassandra/movies$ git commit -m "Primer commit de la carpeta movies con requirements creado"
[main (commit-raiz) 22faecc] Primer commit de la carpeta movies con requirements creado
1 file changed, 5 insertions(+)
Create mode 100644 requirements.txt
john@john-hppavilionlaptop15cw1xxx:~/Desktop/Cassandra/movies$ git push -u origin main
Enumerando objetos: 3, listo.
Contando objetos: 100% (3/3), 322 byte | 322.00 KiB/s, listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (2/2), listo.
Escribiendo objetos: 100% (3/3), 322 byte | 322.00 KiB/s, listo.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/JohnV-1/Class-Activity---Cassandra-Python-Driver.git
 * [new branch]    main -> main
rama 'main' configurada para rastrear 'origin/main'.
john@john-hppavilionlaptop15cw1xxx:~/Desktop/Cassandra/movies$
```

Añadir la carpeta venv al gitignore

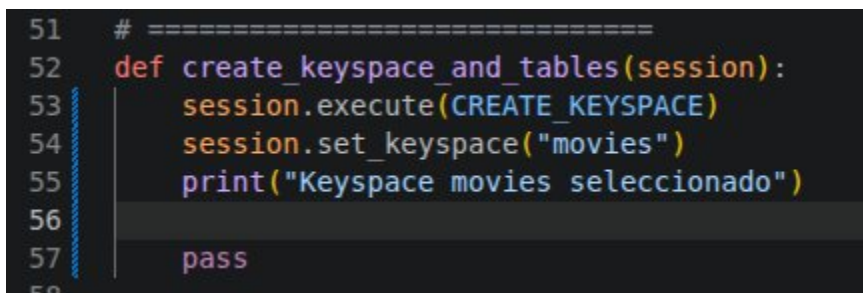


Paso 2: Configurar Cassandra y el Keyspace

Creacion de un keyspace llamado movies.



Tu aplicación de Python deberá conectarse a este keyspace utilizando el Cassandra Python Driver.



Paso 3: Crear tablas para diferentes patrones de acceso

Creación de la tabla 1: movie_by_title

```
9  """
10 ✓ CREATE_TABLE_MOVIE_BY_TITLE = CREATE_TABLE = """
11 CREATE TABLE IF NOT EXISTS movie_by_title (
12     movie_id UUID,
13     title TEXT,
14     release_year INT,
15     genre TEXT,
16     rating FLOAT,
17     director TEXT,
18     PRIMARY KEY (title, release_year)
19 )
20 """
```

Creación de la tabla 2: movie_by_genre

```
✓ CREATE_TABLE_MOVIE_BY_GENRE = """
CREATE TABLE IF NOT EXISTS movie_by_genre (
    movie_id UUID,
    title TEXT,
    release_year INT,
    genre TEXT,
    rating FLOAT,
    director TEXT,
    PRIMARY KEY (genre, rating, movie_id)
)
"""
```

En esta tabla se decidió añadir movie_id como otra clustering key para desempatar cuando las calificaciones sean iguales.

Funcion para crear las tablas dentro del keyspace

```
51 # =====
52 def create_keyspace_and_tables(session):
53     session.execute(CREATE_KEYSPACE)
54     session.set_keyspace("movies")
55     print("Keyspace movies seleccionado")
56     stmt = session.prepare(CREATE_TABLE_MOVIE_BY_TITLE)
57     session.execute(stmt)
58     print("Tabla movie_by_title creada")
59     stmt = session.prepare(CREATE_TABLE_MOVIE_BY_GENRE)
60     session.execute(stmt)
61     print("Tabla movie_by_genre creada")
62     pass
```

Paso 4: Insertar datos de películas en ambas tablas

Consultas para la inserccion de datos:

```
51
52 INSERT_MOVIE_TITLE = """
53 INSERT INTO movie_by_title
54     (movie_id, title, release_year, director,
55      genre, rating)
56 VALUES (?, ?, ?, ?, ?, ?)
57 """
58 INSERT_MOVIE_GENRE = """
59 INSERT INTO movie_by_genre
60     (movie_id, title, release_year, director,
61      genre, rating)
62 VALUES (?, ?, ?, ?, ?, ?)
63 """
```

Funcion para insertar datos

```
✓ def insert_movie(session, title, year, director, genre, rating):
    movie_id = uuid.uuid4()
    stmt = session.prepare(INSERT_MOVIE_TITLE)
    session.execute(stmt, (movie_id, title, year, director, genre, rating))
    stmt = session.prepare(INSERT_MOVIE_GENRE)
    session.execute(stmt, (movie_id, title, year, director, genre, rating))
    pass
```

Paso 5: Consultar los datos utilizando diferentes patrones de acceso

5.1 Consultar por título

Consulta por titulo

```
SELECT_BY_TITLE = "SELECT * FROM movie_by_title WHERE title = ? AND release_year = ?"
```

Funcion para consultar por titulo

```
def query_by_title(session, title, year):
    stmt = session.prepare(SELECT_BY_TITLE)
    rows = session.execute(stmt, (title, year))
    for r in rows:
        print("-----Película-----")
        print(f"Titulo: {r.title}\nAño de estreno: {r.release_year}")
    pass
```

```
=== Movie Database Menu ===
1. Insertar película
2. Consultar por título
3. Consultar por género
4. Actualizar director
5. Eliminar película
0. Salir
Seleccione opción: 2
Título: titanic
Año: 2005
-----Película-----
Titulo: titanic
Año de estreno: 2005
```

5.2 Consultar películas por género

Consulta por genero

```
SELECT_BY_GENRE = "SELECT * FROM movie_by_genre WHERE genre = ? "
```

Funcion para consultar por genero

```
✓ def query_by_genre(session, genre):  
    stmt = session.prepare(SELECT_BY_GENRE)  
    rows = session.execute(stmt, (genre,))  
    print(f"\nGenero: {genre}")  
✓    for r in rows:  
        print(f"Pelicula: {r.title} - {r.rating}")  
    pass
```

```
=== Movie Database Menu ===  
1. Insertar película  
2. Consultar por título  
3. Consultar por género  
4. Actualizar director  
5. Eliminar película  
0. Salir  
Seleccione opción: 3  
Género: action  
  
Genero: action  
Pelicula: jojos - 9.0  
Pelicula: matrix - 8.0  
Pelicula: titanic - 7.0
```

Paso 6: Actualizar información de una película

aplicación

6.1 Intentar actualizar el campo

Explica:

- ¿Fue posible realizar la actualización?

Si

- ¿Por qué?

Porque en Cassandra, la instrucción UPDATE funciona como un upsert (actualiza si existe, inserta si no existe).

- ¿director forma parte de la llave primaria?

No es solo un atributo más de los datos asociados a esa llave.

- ¿Qué información necesita Cassandra para identificar el registro que se desea actualizar?

Cassandra necesita la llave primaria completa y exacta dentro de la cláusula WHERE

Fuentes:

- *UPDATE*. (n.d.). Datastax.com. Retrieved September 14, 2026, from https://docs.datastax.com/en/cql-oss/3.3/cql/cql_reference/cqlUpdate.html

6.2 Actualizar la misma película en ambas tablas

Consulta

```
✓ UPDATE_BY_TITLE="""
UPDATE movie_by_title
SET director = ?
WHERE title = ? AND release_year = ?
"""

✓ UPDATE_BY_GENRE="""
UPDATE movie_by_genre
SET director = ?
WHERE genre = ? AND rating = ? AND movie_id = ?
"""
```

Funcion

```
✓ def update_movie_director(session, title, year, new_director):  
    stmt = session.prepare(SELECT_BY_TITLE)  
    rows = session.execute(stmt, (title, year))  
    ✓ for r in rows:  
        |     print(f"Antiguo director {r.director}")  
  
    stmt = session.prepare(UPDATE_BY_GENRE)  
    session.execute(stmt, (new_director, r.genre, r.rating, r.movie_id))  
    stmt = session.prepare(UPDATE_BY_TITLE)  
    session.execute(stmt, (new_director, title, year))  
    print(f"\nEl director fue actualizado a '{new_director}' en ambas tablas.")  
    pass  
  
✓ def delete_movie(session, title, genre, rating, release_year):  
    stmt = session.prepare(DELETE_MOVIE_TITLE)  
    session.execute(stmt, (title, release_year))  
    stmt = session.prepare(DELETE_MOVIE_GENRE)  
    session.execute(stmt, (genre, rating))  
    print(f"Pelicula: {title} eliminada")  
    pass
```

=== Movie Database Menu ===

1. Insertar película
2. Consultar por título
3. Consultar por género
4. Actualizar director
5. Eliminar película
0. Salir

Seleccione opción: 4

Título: titanic

Año: 2005

Nuevo Director: Daniel

Antiguo director john

El director fue actualizado a 'Daniel' en ambas tablas.

```
=== Movie Database Menu ===
1. Insertar película
2. Consultar por título
3. Consultar por género
4. Actualizar director
5. Eliminar película
0. Salir
Seleccione opción: 4
Título: titanic
Año: 2005
Nuevo Director: john
Antiguo director Daniel

El director fue actualizado a 'john' en ambas tablas.
```

Después, explica:

- **¿Cassandra puede actualizar automáticamente el registro correspondiente en ambas tablas?**

No ya que cada tabla es independiente

- **¿Qué ocurre cuando la misma información se almacena en múltiples tablas?**

Ocurre lo que en Cassandra se llama Desnormalización de datos (Denormalization). Al duplicar la información, se asume un compromiso (Trade-off): las consultas de lectura (SELECT) se vuelven extremadamente rápidas porque toda la información está lista en una sola tabla, pero los procesos de escritura (UPDATE, INSERT, DELETE) se vuelven más complejos porque los datos deben modificarse en cada tabla de forma independiente, corriendo el riesgo de inconsistencia si una actualización falla y la otra no.

- **¿Cómo debe manejar la aplicación las actualizaciones de información desnormalizada?**

Ejecutando múltiples consultas de actualización para garantizar la consistencia de los datos .

- **¿Por qué es importante este comportamiento al diseñar aplicaciones con Cassandra?**

En bases de datos relacionales modelamos los datos según su estructura interna y los relacionamos (Normalización) para ahorrar espacio en disco. En Cassandra, el almacenamiento es barato, pero el procesamiento distribuido (joins) es lento. Por ello, en Cassandra se diseña una tabla por cada consulta que la aplicación va a hacer.

Fuentes:

- *UPDATE*. (n.d.). Datastax.com. Retrieved September 14, 2026, from https://docs.datastax.com/en/cql-oss/3.3/cql/cql_reference/cqlUpdate.html
- *Cassandra - update data*. (n.d.). Tutorialspoint.com. Retrieved September 14, 2026, from https://www.tutorialspoint.com/cassandra/cassandra_update_data.htm

Paso 7: Eliminar los datos y cerrar la conexión

Consulta para eliminar datos

```
DELETE_MOVIE_TITLE = "DELETE FROM movie_by_title WHERE title= ? AND release_year= ?"

DELETE_MOVIE_GENRE = "DELETE FROM movie_by_genre WHERE genre= ? AND rating= ?"
```

Funcion para eliminar

```
def delete_movie(session, title, genre, rating, release_year):
    stmt = session.prepare(DELETE_MOVIE_TITLE)
    session.execute(stmt, (title, release_year))
    stmt = session.prepare(DELETE_MOVIE_GENRE)
    session.execute(stmt, (genre, rating))
    print(f"Pelicula: {title} eliminada")
    pass
```

Cerrar la conexion

```
elif choice == '0':
    # Cerrar conexión y salir
    cluster.shutdown()
    print("Sesión finalizada")
    sys.exit()
    pass
```